

Software Engineering – Lab 3

Name: Madhumitha Dhondi

SRN: PES1UG24AM923

Class: AIML – H

Architectural Justification

Multi-Vendor Artisan E-Commerce Marketplace — Lab 3

Architecture Selection: *We chose Microservices Architecture for the Multi-Vendor Artisan E-Commerce Marketplace.*

Architectural Choice

The system is decomposed into six independently deployable services — an API Gateway, Vendor Storefront Service, Product Catalog Service, Cart & Checkout Service, Payment Splitting Service, and Order Tracking & Notification Service — each mapping directly to one of our Lab 1 functional requirements (FR-001 through FR-005).

Two Reasons for This Choice

1. Wildly different scaling needs across concerns: Product Catalog traffic is dominated by high-volume, read-heavy browsing (many shoppers viewing listings), while Cart & Checkout and Payment Splitting are comparatively low-volume but transactional and correctness-critical. A Layered or monolithic design would force all of this to scale together; Microservices lets us scale the Catalog Service horizontally to meet browsing demand without over-provisioning the payment path.

2. Independent deployment and fault isolation for a multi-tenant platform: Vendor onboarding features, catalog search improvements, and payment/compliance updates are independently evolving concerns that plausibly ship on different schedules. Microservices lets each be deployed without redeploying the whole platform, and if the Order Tracking & Notification Service goes down, Checkout can still complete — a failure mode a monolith or tightly-coupled Layered design wouldn't isolate as cleanly.

Security Advantage

The Payment Splitting Service is isolated as its own service boundary, reachable only from Cart & Checkout rather than directly from the API Gateway or any customer-facing surface. This keeps payment-adjacent logic out of the broader storefront and catalog codebases and narrows the scope of infrastructure that needs to meet PCI-DSS compliance (our Lab 1 NFR-002) to a single, dedicated service rather than the whole platform.

Performance Benefit

The Product Catalog Service can be scaled independently — additional read replicas, caching, or CDN in front of it — to meet our Lab 1 NFR-001 target of under 500ms image/catalog load times, without needing to scale or redeploy the Cart & Checkout or Payment Splitting services alongside it. In a monolithic or Layered design, meeting that latency target under peak load would mean scaling the entire application, including code paths that don't need it.

Component Diagram: Self-Service Coffee Kiosk System

Layered Architecture

